

Opt

优化

前言

DFS（深度优先搜索）是一种常见的算法，大部分的题目都可以用 DFS 解决，但是大部分情况下，这都是骗分算法，很少会有爆搜为正解的题目。因为 DFS 的时间复杂度特别高。（没学过 DFS 的请自行补上这一课）

既然不能成为正解，那就多骗一点分吧。那么这一篇文章将介绍一些实用的优化算法（俗称「剪枝」）。

先来一段深搜模板，之后的模板将在此基础上进行修改。

```
1 int ans = 最坏情况, now; // now 为当前答案
2
3 void dfs(传入数值) {
4     if (到达目的地) ans = 从当前解与已有解中选最优;
5     for (遍历所有可能性)
6         if (可行) {
7             进行操作;
8             dfs(缩小规模);
9             撤回操作;
10    }
11 }
```

其中的 ans 可以是解的记录，那么从当前解与已有解中选最优就变成了输出解。

剪枝方法

最常用的剪枝有三种，记忆化搜索、最优性剪枝、可行性剪枝。

记忆化搜索

因为在搜索中，相同的传入值往往会带来相同的解，那我们就可以用数组来记忆，详见 [记忆化搜索](#)。

模板：

```

1 int g[MAXN]; // 定义记忆化数组
2 int ans = 最坏情况, now;
3
4 void dfs f(传入数值) {
5     if (g[规模] != 无效数值) return; // 或记录解, 视情况而定
6     if (到达目的地) ans = 从当前解与已有解中选最优; // 输出解, 视情况而定
7     for (遍历所有可能性)
8         if (可行) {
9             进行操作;
10            dfs(缩小规模);
11            撤回操作;
12        }
13 }
14
15 int main() {
16     // ...
17     memset(g, 无效数值, sizeof(g)); // 初始化记忆化数组
18     // ...
19 }

```

最优性剪枝

在搜索中导致运行慢的原因还有一种, 就是在当前解已经比已有解差时仍然在搜索, 那么我们只需要判断一下当前解是否已经差于已有解。

模板:

```

1 int ans = 最坏情况, now;
2
3 void dfs(传入数值) {
4     if (now比ans的答案还要差) return;
5     if (到达目的地) ans = 从当前解与已有解中选最优;
6     for (遍历所有可能性)
7         if (可行) {
8             进行操作;
9             dfs(缩小规模);
10            撤回操作;
11        }
12 }

```

可行性剪枝

在搜索过程中当前解已经不可用了还继续搜索下去也是运行慢的原因。

模板:

```

1 int ans = 最坏情况, now;
2
3 void dfs(传入数值) {
4     if (当前解已不可用) return;
5     if (到达目的地) ans = 从当前解与已有解中选最优;
6     for (遍历所有可能性)
7         if (可行) {
8             进行操作;
9             dfs(缩小规模);
10            撤回操作;
11        }
12 }

```

剪枝思路

剪枝思路有很多种，大多需要对于具体问题来分析，在此简要介绍几种常见的剪枝思路。

- 极端法：考虑极端情况，如果最极端（最理想）的情况都无法满足，那么肯定实际情况搜出来的结果不会更优了。
- 调整法：通过对子树的比较剪掉重复子树和明显不是最有「前途」的子树。
- 数学方法：比如在图论中借助连通分量，数论中借助模方程的分析，借助不等式的放缩来估计下界等等。

例题

▼ 工作分配问题

题目描述

有 n 份工作要分配给 m 个人来完成，每个人完成一份。第 i 个人完成第 j 份工作所用的时间为一个正整数 t_{ij} ，其中 $1 \leq i \leq m, 1 \leq j \leq n$ 。试确定一个分配方案，使得完成这 n 份工作的时间总和最小。

输入包含 m 行。

第 1 行为一个正整数 n 。

第 2 行到第 m 行中每行都包含 n 个正整数，形成了一个 $m \times n$ 的矩阵。在该矩阵中，第 i 行第 j 列元素 t_{ij} 表示第 i 个人完成第 j 件工作所要用的时间。

输出包含一个正整数，表示所有分配方案中最小的时间总和。

数据范围

输入样例

```
1 5
2 9 2 9 1 9
3 1 9 8 9 6
4 9 9 9 9 1
5 8 8 1 8 4
6 9 1 7 8 9
```

输出样例

```
1 5
```

由于每个人都必须分配到工作，在这里可以建一个二维数组 $time[i][j]$ ，用以表示 i 个人完成 j 号工作所花费的时间。给定一个循环，从第 1 个人开始循环分配工作，直到所有人都分配到。为第 i 个人分配工作时，再循环检查每个工作是否已被分配，没有则分配给 i 个人，否则检查下一个工作。可以用一个一维数组 $is_working[j]$ 来表示第 j 号工作是否已被分配，未分配则 $is_working[j]=0$ ，否则 $is_working[j]=1$ 。利用回溯思想，在工人循环结束后回到上一工人，取消此次分配的工作，而去分配下一工作直到可以分配为止。这样，一直回溯到第 1 个工人后，就能得到所有的可行解。

检查工作分配，其实就是判断取得可行解时的二维数组的第一维下标各不相同并且第二维下标各不相同。而我们要得到完成这 n 份工作的最小时间总和，即可行解中时间总和最小的一个，故需要再定义一个全局变量

$cost_time_total_min$ 表示目前找到的解中最小的时间总和，初始 $cost_time_total_min$ 为 $time[i][i]$ 之和，即对角线

工作时间相加之和。在所有人分配完工作时，比较 `count` 与 `cost_time_total_min` 的大小，如果 `count` 小于 `cost_time_total_min`，说明找到了一个最优解，此时就把 `count` 赋给 `cost_time_total_min`。

但考虑到算法的效率，这里还有一个剪枝优化的工作可以做。就是在每次计算局部费用变量 `count` 的值时，如果判断 `count` 已经大于 `cost_time_total_min`，就没必要再往下分配了，因为这时得到的解必然不是最优解。

► 参考代码

本页面最近更新：2023/3/22 15:46:23, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[abc1763613206](#), [AngelKitty](#), [CBW2007](#), [ChungZH](#), [lr1d](#), [CCXXXI](#), [Chrogeek](#), [countercurrent-time](#), [Enter-tainer](#), [H-J-Granger](#), [kenlig](#), [ksyx](#), [Marcythm](#), [mgt](#), [NachtgeistW](#), [ouuan](#), [ouuan](#), [sshwy](#), [StudyingFather](#), [SukkaW](#), [TrisolarisHD](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用